NCCLbpf: Verified, Composable Policy Execution for GPU Collective Communication

Anonymous Author(s)

ABSTRACT

NCCL is the de facto standard for collective GPU communication in large-scale distributed training, relying heavily on plugins to customize runtime behavior. However, these plugins execute as unverified native code within NCCL's address space, risking job crashes, silent state corruption, and downtime from restarts during policy updates. Inspired by kernel extensibility models, we introduce NCCLbpf, a verified, high-performance extension framework embedding a userspace eBPF runtime directly into NCCL's existing plugin interfaces, without modifying NCCL itself. NCCLbpf offers load-time static verification to prevent unsafe plugin execution, structured cross-plugin maps enabling composable policies and closed-loop adaptation, and atomic policy hot-reloads eliminating downtime previously required for policy updates. Evaluations on 8× NVIDIA B300 GPUs connected via NVLink demonstrate that NCCLbpf imposes just 80–130 ns overhead per tuner decision (less than 0.03% of collective latency), prevents all tested unsafe plugin behaviors at load-time, and enables a message-size-aware eBPF policy that improves AllReduce throughput by 5–27% over NCCL's default in the 4–128 MiB range.

1 INTRODUCTION

Collective GPU communication performance is crucial for large-scale distributed training frameworks such as Megatron-LM [15], DeepSpeed [13], and PyTorch FSDP [20], all of which depend on NCCL [10] for efficient inter-GPU data movement. The performance of NCCL collectives (e.g., AllReduce, AllGather) significantly depends on runtime decisions: algorithm selection (ring vs. tree), transport protocols (LL vs. Simple [7]), and parallelization channels. Recognizing this dependency, NCCL exposes a modular plugin architecture, including tuner, profiler, and net plugins, allowing cloud operators and hardware vendors to tailor policies to match specific deployment characteristics. Major providers already leverage these plugins extensively in production scenarios [18].

However, this flexibility comes at substantial reliability and operability risks. NCCL plugins execute as native code within NCCL's runtime environment without sandboxing or verification. A single null-pointer dereference can crash a critical GPU training job; an infinite loop or race condition can silently halt collective operations; and subtle memory corruption can cause unpredictable performance degradation or safety violations. NCCL's own inspector plugin has caused production training crashes due to use-after-free and deadlock bugs [11], and profiler plugins have triggered segfaults on multi-GPU nodes [12]. Moreover, NCCL plugins lack structured state-sharing capabilities, making closed-loop policy adjustments across plugins impossible. Updating plugin logic requires restarting affected jobs, resulting in disruptive downtime and hindering rapid policy iteration in production environments.

We identify a structural analogy between NCCL plugin extension points and kernel-level hooks, where the eBPF framework [17] has already successfully addressed similar challenges [19, 23, 24]. In both contexts, extension points must execute safely and efficiently in performance-critical environments. NCCL's net plugin closely mirrors eBPF's original packet-processing use case, and NCCL's tuner and profiler interfaces correspond to kernel tracepoints where lightweight policy-driven decisions influence critical runtime behavior. Inspired by eBPF's verified extensibility model, we present NCCLbpf, which integrates a verified, userspace eBPF runtime [22] directly within NCCL's existing plugin interface, without modifying NCCL source code.

NCCLbpf provides three properties: (1) load-time static verification guarantees safety before execution, preventing crashes and state corruption; (2) structured, typed maps enable composable, cross-plugin state sharing, enabling previously impossible closed-loop adaptations; and (3) atomic policy hot-reload capability allows operators to update policies seamlessly at runtime without interruption.

We implement NCCLbpf atop NCCL version 2.29.7 using bpf-time, a lightweight userspace eBPF runtime. Our evaluation on 8× NVIDIA B300 GPUs with NVLink shows that NCCLbpf adds 80–130 ns overhead per policy decision, representing less than 0.03% of collective latency. Atomic updates incur a downtime of just 1.07 µs and introduce zero dropped calls across 400,000 policy invocations. A message-size-aware eBPF policy improves 8-GPU AllReduce throughput by 5–27% over NCCL's default algorithm selection in the 4–128 MiB range.

This paper makes three contributions:

(1) **NCCLbpf**, the first framework to bring verified, composable eBPF policy execution to NCCL's plugin architecture, transparently enhancing safety and extensibility without modifying NCCL source code (§3–4).
(2) A structured, typed-map-based composability mechanism enabling previously isolated NCCL plugins to coordinate and dynamically adapt, facilitating closed-loop policy control (§3, §5.3).
(3) An evaluation on 8× NVIDIA B300 NVLink GPUs demonstrating low overhead (80–130 ns per decision), verified runtime safety (rejecting all unsafe plugins tested), and a message-size-aware policy that improves AllReduce throughput by up to 27% (§5).

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference'17, July 2017, Washington, DC, USA
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

2 BACKGROUND AND MOTIVATION

NCCL and its plugin system. NCCL [10] implements GPU collective primitives (AllReduce, AllGather, Broadcast, etc.) optimized for multi-GPU interconnects. For each collective call, the *tuner* plugin's `getCollInfo` receives the collective type, message size, and rank topology, and selects an algorithm (ring, tree), protocol (LL, LL128, Simple [7]), and channel count by modifying a cost table. The *profiler* plugin receives timestamped event callbacks; the *net* plugin controls network transport; and the *env* plugin sets runtime parameters. All four are loaded via `dlopen` as shared libraries. The tuner API has evolved rapidly, from v2 (simple integer outputs) to v5 (in-place 2D float cost table modification) in two years, and major cloud providers ship custom tuner plugins [18]. Despite this growing ecosystem, plugins remain independent extension points with no cross-plugin communication and no safety verification.

The safety gap. Native plugins execute with full process privileges. NCCL's own inspector plugin has caused production training crashes due to use-after-free and deadlock bugs [11], and profiler plugins have triggered segfaults on multi-GPU nodes [12]. At scale, such failures are costly: Llama 3 pre-training on 16,384 GPUs experienced 466 job interruptions in 54 days [4], and analyses of production GPU clusters find that ~30% of training jobs fail, with programming and configuration errors as the dominant cause [9]. The absence of safe hot-reload further compounds the risk: rolling out plugin updates across a fleet requires restarting every training job.

eBPF as safe extension mechanism. Extended BPF (eBPF) [17] is an execution framework, originally implemented in the Linux kernel for packet filtering and tracing, that statically verifies programs for memory safety and bounded execution before JIT-compiling them to native code. eBPF has been adopted as a safe-extension mechanism across storage [23], networking [2, 24], memory management [19], and GPU driver-level resource management [21]. Recent userspace runtimes [22] extend its reach beyond the kernel, enabling eBPF execution inside application processes without kernel privileges. Three properties make eBPF well-suited to NCCL policy execution: (1) load-time verification catches memory-safety and termination bugs that cause the crashes described above; (2) typed maps provide concurrent state sharing between profiler and tuner without ad hoc shared memory; and (3) atomic program replacement enables verified hot-reload.

3 DESIGN

3.1 Design Tensions

T1: Safety vs. overhead. NCCLbpf verifies only at load time (1–5 ms, amortized over the job lifetime), relying on the invariant that verified BPF bytecode, once JIT-compiled, cannot violate its safety guarantees at runtime. The tradeoff: verification catches memory safety and termination bugs, not semantic correctness.

T2: Structured sharing vs. flexibility. All cross-component state sharing uses typed eBPF maps with atomic access semantics. Maps constrain policies to fixed-size keys/values, but NCCL policy state is inherently simple (per-communicator scalars), and structured sharing eliminates the locking bugs that plague ad hoc native plugin state sharing.

T3: Availability vs. consistency during updates. Hot-reload atomically swaps the function pointer; any in-progress call completes under the old policy, the next call uses the new one. No call is lost. If the new policy fails verification, the old policy continues. The tradeoff: different threads may briefly execute different policies, which is acceptable because collective decisions are independent across calls.

T4: Compatibility vs. capability. Our system operates within NCCL's existing plugin ABI, requiring no source modifications, no custom builds, and no kernel modules. The action space is limited to algorithm, protocol, and channel count selection. Deeper interventions are outside scope, but the exposed knobs cover the most impactful tuning decisions. NCCLbpf selects among NCCL's built-in algorithms; it does not implement custom collective algorithms.

3.2 Threat Model

Policies are written by platform operators, not untrusted tenants. The eBPF verifier (PREVAIL-based) checks memory safety, bounded execution, stack safety, and helper whitelisting. The system does *not* protect against resource exhaustion, side channels, bugs in the trusted computing base (bpftime JIT, PREVAIL verifier, host plugin), or semantically incorrect decisions. This provides defense in depth (eliminating crashes, hangs, and memory corruption), analogous to the Linux kernel's eBPF model.

3.3 Architecture

Figure 1 shows the architecture. Policy authors write restricted C compiled to BPF ELF objects. At load time, each program is verified and JIT-compiled to x86-64 code. The map subsystem provides intra-policy state and cross-plugin communication: a profiler eBPF program writes latency observations; the tuner reads them. Hot-reload verifies and JIT-compiles a replacement, then atomically swaps the function pointer via compare-and-swap.

Policy programming model. A policy takes a `policy_context` (collective type, message size, rank count) and writes algorithm, protocol, and channel count decisions. The verifier ensures policies only read input fields and write output fields. Map lookups require null checks before dereference; omitting one causes load-time rejection.

Listing 1: Profiler-to-tuner closed loop. The profiler (top) writes latency into a shared eBPF map; the tuner (bottom) reads it for adaptive channel selection. Null checks (lines 5, 14) are enforced by the verifier.

```
1  /* --- profiler eBPF program --- */
2  SEC("profiler")
3  int record_latency(struct profiler_context *ctx) {
4    __u32 key = ctx->comm_id;
5    struct latency_state *st =
6      bpf_map_lookup_elem(&latency_map, &key);
7    if (!st) return 0;
8    st->avg_latency_ns = ctx->latency_ns;
9    st->channels = ctx->n_channels;
10   return 0;
11 }
12 /* --- tuner eBPF program --- */
```

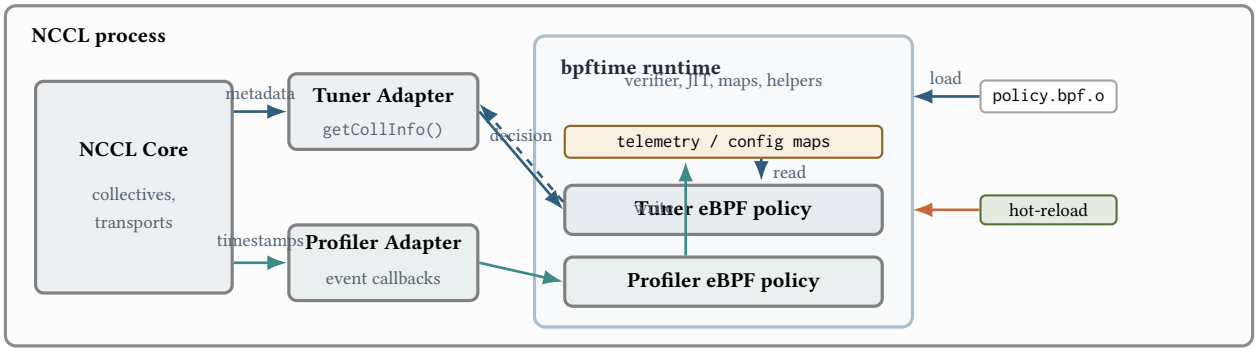

Figure 1: NCCLbpf architecture. Policy programs are verified via PREVAIL, JIT-compiled by bpftime, and execute inside the NCCL process. Profiler and tuner communicate through shared typed eBPF maps. The entire system loads as standard NCCL plugins.

```
13  SEC("tuner")
14  int size_aware_adaptive(struct policy_context *ctx) {
15    __u32 key = ctx->comm_id;
16    struct latency_state *st =
17      bpf_map_lookup_elem(&latency_map, &key);
18    if (!st) { ctx->n_channels = 4; return 0; }
19    if (ctx->msg_size <= 32 * 1024)
20      ctx->algorithm = NCCL_ALGO_TREE;
21    else
22      ctx->algorithm = NCCL_ALGO_RING;
23    ctx->protocol = NCCL_PROTO_SIMPLE;
24    if (st->avg_latency_ns > 1000000)
25      ctx->n_channels = min(st->channels + 1, 16);
26    else
27      ctx->n_channels = st->channels;
28    return 0;
29  }
```

The two programs share `latency_map` via the eBPF map subsystem, a capability absent from NCCL's native plugin architecture, where tuner and profiler have no shared state. Null checks on map lookups (lines 5 and 14) are enforced by the verifier; omitting either causes load-time rejection.

4 IMPLEMENTATION

NCCLbpf is implemented in approximately 2,500 lines of C/C++ (plugin host) and 500 lines of restricted C (policy programs).

bpftime integration. We embed bpftime [22] as the userspace eBPF runtime. bpftime provides an LLVM-based JIT compiler, a PREVAIL-based [5] static verifier, and a map subsystem. The verifier runs in the same address space as the plugin, adding approximately 1–5 ms of one-time startup overhead amortized over the lifetime of the training job. The LLVM JIT produces optimized x86-64 code, narrowing the gap to native performance.

Plugin registration. NCCLbpf registers itself as both a tuner and profiler plugin via NCCL's `ncclTunerPlugin_v3` (or v5) and `ncclProfilerPlugin_v1` (or v6) interfaces. The tuner plugin's `getCollInfo()` function constructs a `policy_context`, invokes the JIT-compiled eBPF function, and copies the output fields into NCCL's result structure. The profiler plugin's event callbacks extract latency information and update the shared eBPF maps.

NCCL integration challenges. NCCL's tuner API uses cost arrays rather than direct algorithm IDs: the tuner sets costs to zero for preferred choices and to a sentinel value (`1e9`) for others, allowing NCCL to fall back gracefully if the requested combination is unavailable. Our integration translates eBPF policy outputs into cost array entries. NCCL also passes a maximum channel count that the tuner must respect; our native baseline layer clamps the policy's request. Communicator identification required deriving a stable ID from the context pointer via hashing, since the communicator ID is not directly exposed in the tuner API as a simple integer.

Hot-reload mechanism. The active policy is stored as an atomic function pointer. Reload has three phases: (1) verify the new BPF program, (2) JIT-compile it, (3) atomic compare-and-swap on the pointer. The total reload cost is ~9.4 ms; only the final swap (1.07 µs) touches the hot path. The old pointer is retained until in-flight calls drain. If the replacement fails verification, the swap is aborted and the old policy continues; the system never enters an unverified state.

Native baseline for comparison. To provide a fair overhead comparison, we implemented a native C++ baseline that performs identical policy logic without the eBPF layer, compiled with the same optimization flags (`-O2`). The overhead difference between the native baseline and eBPF policies directly measures the cost of the eBPF dispatch and JIT execution layers, isolated from any policy logic cost.

5 EVALUATION

Testbed. CPU microbenchmarks run on a 240-core AMD EPYC 9575F; each benchmark performs 1 million calls and reports P50/P99 latencies. GPU experiments use 8× NVIDIA B300 SXM6 GPUs (Blackwell, 275 GB each) connected via NVLink 5 (NV18, 1.8 TB/s per GPU) with CUDA 13.0 and NCCL 2.29.7.

5.1 Overhead

Table 1 reports per-call latency. The overhead is predictable: approximately $80 + 30n_{\text{lookup}} + 10n_{\text{update}}$ ns. Even the most complex policy (`slo_enforcer`) adds only 130 ns. For a 128 MiB 8-GPU AllReduce on NVLink (~394 µs), this is 0.03% of collective latency.

End-to-end GPU measurements on NVLink across small message sizes (8 B to 256 KiB) show that the noop plugin adds ~1.3 µs of fixed overhead (~4% of the 31 µs NVLink baseline). This overhead

Table 1: CPU microbenchmark (1M calls each). eBPF policies add 80–130 ns over native code. The overhead decomposes into eBPF dispatch (+80 ns), map lookup (+30 ns each), and map update (+10 ns each).

Policy	P50 (ns)	P99 (ns)	ΔP50
Native baseline	20	30	—
`noop`	100	111	+80
`size_aware_v2`	100	111	+80
`lookup_only`	130	140	+110
`lookup_update`	140	151	+120
`adaptive_channels`	140	151	+120
`slo_enforcer`	150	160	+130

comes from the plugin framework (shared memory setup, cost table writes), not the eBPF dispatch itself (33 ns). For messages of 4 MiB and above, the overhead is below measurement noise (<0.1%).

5.2 Safety and Hot-Reload

We tested 14 eBPF programs against the PREVAIL-based verifier: 7 safe policies (including all policies in Table 1) were accepted; 7 unsafe programs, each targeting a distinct bug class (null-pointer dereference, out-of-bounds access, illegal helper, stack overflow, unbounded loop, input-field write, division by zero), were rejected at load time with actionable error messages.

The safety difference is concrete. We implemented the same null-dereference bug as both a native C plugin and an eBPF policy:

```
Native plugin: Signal: SIGSEGV (address 0x0)
                in getCollInfo() at native_bad_plugin.so
eBPF policy:   VERIFIER REJECT: R0 is a pointer to
                map_value_or_null; must check != NULL
                before dereference at insn 7
```

The native plugin crashes the training job; the eBPF policy is caught before execution.

Hot-reload swaps the active policy atomically via compare-and-swap (1.07 µs swap time; ~9.4 ms including verification and JIT). Across 400,000 continuous invocations, we observe zero lost calls. If the replacement fails verification, the old policy continues uninterrupted.

5.3 Case Studies

NVLink-aware adaptive policy. On our 8× B300 NVLink testbed, NCCL 2.29.7 defaults to the NVLS algorithm (NVLink SHARP with hardware multicast) for all message sizes. A parameter sweep reveals that Ring with 32 channels outperforms NVLS by 5–27% in the 4–128 MiB range, while NVLS is superior at 256 MiB and above (Table 2).

Static environment variables (`NCCL_ALGO=Ring`) cannot exploit this: they apply globally and would degrade the 256 MiB+ range. An eBPF policy can select per-message-size. Our `nvlink_ring_mid_v2` policy (17 lines of C) selects Ring/LL128 for 4–32 MiB, Ring/Simple for 64–192 MiB, and defers to NCCL's default otherwise.

Figure 2 shows the result. The eBPF policy achieves 5.5–26.5% improvement over NCCL default in the targeted range (steady-state, reps 4–5 of 5), while matching default performance outside this

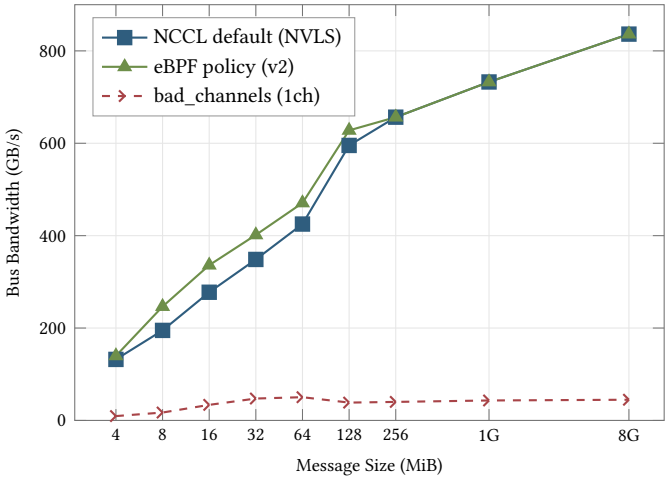

Figure 2: 8-GPU AllReduce on NVLink. The eBPF policy selects Ring/LL128 for 4–32 MiB and Ring/Simple for 64–192 MiB, improving throughput by 5–27% over NCCL's default NVLS algorithm. A deliberately bad policy (1 channel) shows that policies have real control, including destructive power.

Table 2: Algorithm sweep: 8-GPU AllReduce bus bandwidth (GB/s). Ring outperforms NVLS by 5–27% in the 4–128 MiB range.

Size	Default (NVLS)	Ring	Δ
4 MiB	133.5	148.1	+10.9%
8 MiB	196.3	249.7	+27.2%
16 MiB	278.8	337.4	+21.0%
32 MiB	349.3	402.4	+15.2%
64 MiB	425.2	471.8	+11.0%
128 MiB	596.9	628.9	+5.4%
256 MiB	656.5	632.5	−3.7%
8 GiB	836.3	697.6	−16.6%

range. The `noop` policy (not shown) produces identical throughput to the no-plugin baseline, confirming zero overhead at 4 MiB+.

A deliberately bad policy (`bad_channels`, forcing 1 channel) passes the verifier (it is memory-safe) but causes 87–93% throughput degradation, demonstrating that policies have real control over NCCL's behavior. The verifier prevents crashes and memory corruption, not poor decisions; semantic validation remains the operator's responsibility.

Stability. Over 20 independent runs of 8-GPU AllGather at 128 MiB, the default configuration achieves 565.6 ± 0.9 GB/s (CV = 0.15%) and the eBPF v2 policy achieves 565.5 ± 0.6 GB/s (CV = 0.10%). Both configurations are highly stable on NVLink, with the policy showing 32% lower variance. The default has one 3.4σ outlier (562.6 GB/s); the policy distribution is unimodal with no comparable outlier.

Profiler-to-tuner composability. To evaluate cross-plugin composability, we construct an adaptive channels policy that starts with a

conservative channel count (nChannels = 2) and relies on profiler telemetry to adapt upward. Without the profiler, the tuner receives no samples and remains at 2 channels. With the profiler enabled, latency samples flow into a shared eBPF map, and the tuner ramps to the optimal channel count. This validates NCCLbpf's composability model: two independently deployed eBPF programs cooperate through shared typed maps, a capability absent from NCCL's native plugin architecture.

Net plugin extensibility. We implement an eBPF-wrapped net plugin that interposes on NCCL's Socket transport. The wrapper forwards all transport operations to the built-in backend while executing a BPF program at each `isend`/`irecv` call to count bytes and connections via a shared eBPF map. The wrapped transport adds less than 2% overhead, confirming that eBPF hooks can operate on NCCL's data-plane path with negligible cost, paralleling eBPF's established role in kernel packet processing.

6 RELATED WORK

Collective communication. MSCCL [3] and TACCL [14] synthesize static schedules offline; they do not address runtime policy selection or safety. AutoCCL [18] automates tuning via search but uses native code without verification. MSCCL++ [8] provides GPU-driven primitives at the mechanism layer. Hu et al. [7] provide a comprehensive analysis of NCCL's protocol variants and algorithms. gpu_ext [21] applies eBPF to GPU driver-level resource management; NCCLbpf targets the collective communication library layer, where policies govern algorithm and protocol selection rather than device resources. NCCLbpf is complementary: a policy can activate synthesized schedules or tuned configurations based on runtime conditions.

Alternative extension mechanisms. WebAssembly [6] provides sandboxed execution but lacks eBPF's static safety guarantees (termination, bounded memory access), and incurs higher overhead from runtime bounds checks. Out-of-process services add microseconds of IPC latency, an order of magnitude more than NCCLbpf's 80–130 ns overhead. Declarative DSLs cannot express stateful policies or feedback loops. Hardware isolation (Intel MPK [16]) provides memory separation without verification. eBPF uniquely combines static verification with near-native JIT performance in a single model.

7 DISCUSSION

NCCLbpf currently covers tuner, profiler, and net plugins; extending coverage to the env plugin is straightforward. Our net plugin prototype wraps the built-in Socket backend; deeper integration (e.g., RDMA-aware policies) is future work. Our evaluation uses 8 GPUs on a single node with NVLink; multi-node experiments with InfiniBand and larger rank counts are needed to validate the approach at production scale. A higher-level policy DSL compiled to BPF would lower the barrier for ML engineers who are not familiar with eBPF programming.

The eBPF-for-extensions pattern may generalize beyond NCCL. AMD's RCCL [1] exposes a similar plugin architecture, suggesting that a cross-vendor port may be feasible. Other libraries with native-code extension points (MPI implementations, RDMA middleware) face similar safety gaps and could benefit from the same verify-then-execute approach.

8 CONCLUSION

We presented NCCLbpf, a system that transforms NCCL's unsafe native plugin extension points into verified, composable policy hooks by embedding a userspace eBPF runtime. NCCL's plugin architecture is structurally analogous to kernel extension points where eBPF is already proven; NCCLbpf brings this model to GPU collective communication, covering tuner, profiler, and net plugins. NCCLbpf adds 80–130 ns per policy decision and negligible overhead on the net data path, catches crash-inducing bugs at load time, supports atomic hot-reload without call loss, and enables a message-size-aware policy that improves 8-GPU NVLink AllReduce throughput by 5–27% over NCCL's default, with zero NCCL modifications.

REFERENCES

[1] AMD. 2024. RCCL: ROCm Communication Collectives Library. https://github.com/ROCm/rccl.
[2] Zhongjie Chen, Qingkai Meng, ChonLam Lao, Yifan Liu, Fengyuan Ren, Minlan Yu, and Yang Zhou. 2025. eTran: Extensible Kernel Transport with eBPF. In *Proceedings of the 22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI)*.
[3] Meghan Cowan, Saeed Maleki, Madanlal Musuvathi, Olli Saarikivi, and Yifan Xiong. 2023. MSCCLang: Microsoft Collective Communication Language. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*.
[4] Abhimanyu Dubey et al. 2024. The Llama 3 Herd of Models. *arXiv preprint arXiv:2407.21783* (2024).
[5] Elazar Gershuni, Nadav Amit, Arie Gurfinkel, Nina Narodytska, Jorge A. Navas, Noam Rinetzky, Leonid Ryzhyk, and Mooly Sagiv. 2019. Simple and Precise Static Analysis of Untrusted Linux Kernel Extensions. In *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*.
[6] Andreas Haas, Andreas Rossberg, Derek L. Schuff, Ben L. Titzer, Michael Holman, Dan Gohman, Luke Wagner, Alon Zakai, and JF Bastien. 2017. Bringing the Web up to Speed with WebAssembly. In *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*.
[7] Zhiyi Hu, Siyuan Shen, Tommaso Bonato, Sylvain Jeaugey, et al. 2025. Demystifying NCCL: An In-depth Analysis of GPU Communication Protocols and Algorithms. *arXiv preprint arXiv:2507.04786* (2025).
[8] Changho Hwang, Peng Cheng, Roshan Dathathri, Abhinav Jangda, Saeed Maleki, Madan Musuvathi, Olli Saarikivi, Aashaka Shah, Ziyue Yang, Binyang Li, et al. 2026. MSCCL++: Rethinking GPU Communication Abstractions for AI Inference. In *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*.
[9] Myeongjae Jeon, Shivaram Venkataraman, Amar Phanishayee, Junjie Qian, Wencong Xiao, and Fan Yang. 2019. Analysis of Large-Scale Multi-Tenant GPU Clusters for DNN Training Workloads. In *Proceedings of the 2019 USENIX Annual Technical Conference (ATC)*.
[10] NVIDIA. 2024. NCCL: NVIDIA Collective Communications Library. https://github.com/NVIDIA/nccl.
[11] NVIDIA. 2026. Deadlock/crash due to UAF in inspector plugin. https://github.com/NVIDIA/nccl/issues/2000.
[12] NVIDIA. 2026. Inspector bug: segfault encountered during training. https://github.com/NVIDIA/nccl/issues/1992.
[13] Jeff Rasley, Samyam Rajbhandari, Olatunji Ruwase, and Yuxiong He. 2020. DeepSpeed: System Optimizations Enable Training Deep Learning Models with Over 100 Billion Parameters. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*.
[14] Aashaka Shah, Vijay Chidambaram, Meghan Cowan, Saeed Maleki, Madan Musuvathi, Todd Mytkowicz, Jacob Nelson, Olli Saarikivi, and Rachee Singh. 2023. TACCL: Guiding Collective Algorithm Synthesis using Communication Sketches. In *Proceedings of the 20th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*.
[15] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. 2019. Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism. *arXiv preprint arXiv:1909.08053* (2019).

[16] Anjo Vahldiek-Oberwagner, Eslam Elnikety, Nuno O. Duarte, Michael Sammler, Peter Druschel, and Deepak Garg. 2019. ERIM: Secure, Efficient In-Process Isolation with Protection Keys (MPK). In *Proceedings of the 28th USENIX Security Symposium*.

[17] Marcos A. M. Vieira, Matheus S. Castanho, Racyus D. G. Pacífico, Elerson R. S. Santos, Eduardo P. M. Câmara Júnior, and Luiz F. M. Vieira. 2020. Fast Packet Processing with eBPF and XDP: Concepts, Code, Challenges, and Applications. *Comput. Surveys* 53, 1 (2020).

[18] Guanbin Xu, Zhihao Le, Yinhe Chen, Zhiqi Lin, Zewen Jin, Youshan Miao, and Cheng Li. 2025. AutoCCL: Automated Collective Communication Tuning for Accelerating Distributed and Parallel DNN Training. In *Proceedings of the 22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI)*.

[19] Anil Yelam, Kan Wu, Zhiyuan Guo, Suli Yang, Rajath Shashidhara, Wei Xu, Stanko Novaković, Alex C. Snoeren, and Kimberly Keeton. 2025. PageFlex: Flexible and Efficient User-space Delegation of Linux Paging Policies with eBPF. In *Proceedings of the 2025 USENIX Annual Technical Conference (ATC)*.

[20] Yanli Zhao, Andrew Gu, Rohan Varma, Liang Luo, Chien-Chin Huang, Min Xu, Les Wright, Hamid Shojanazeri, Myle Ott, Sam Shleifer, Alban Desmaison, Can Balioglu, Pritam Damania, Benjamin Nguyen, Geeta Chauhan, Yuchen Hao, Ajit Mathews, and Shen Li. 2023. PyTorch FSDP: Experiences on Scaling Fully Sharded Data Parallel. *Proceedings of the VLDB Endowment* 16, 12 (2023).

[21] Yusheng Zheng, Tong Yu, Yiwei Yang, et al. 2025. gpu_ext: Extensible OS Policies for GPUs via eBPF. *arXiv preprint arXiv:2512.12615* (2025).

[22] Yusheng Zheng, Tong Yu, Yiwei Yang, Yanpeng Hu, Xiaozheng Lai, Dan Williams, and Andi Quinn. 2025. Extending Applications Safely and Efficiently. In *Proceedings of the 19th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*.

[23] Yuhong Zhong, Haoyu Li, Yu Jian Wu, Ioannis Zarkadas, Jeffrey Tao, Evan Mesterhazy, Michael Makris, Junfeng Yang, Amy Tai, Ryan Stutsman, and Asaf Cidon. 2022. XRP: In-Kernel Storage Functions with eBPF. In *Proceedings of the 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*.

[24] Yang Zhou, Zezhou Wang, Sowmya Dharanipragada, and Minlan Yu. 2023. Electrode: Accelerating Distributed Protocols with eBPF. In *Proceedings of the 20th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*.